Comparativa de consumo de API con Volley, OkHttp y Retrofit

Tarea 2.5 · Trabajo individual

Estudiante	José Fernando Orellana Reyes
Número de cuenta	202210010352
Repositorio	Kirklen0910/Exa1Pa1199
Rama	feature/comparativa-fernando-orellana
Pantalla	ComparativaFernandoActivity
Endpoint principal	GET /api/registros

Informe generado a partir de la implementación y de las mediciones realizadas en el emulador Pixel 8 (Android 15).

1. Resumen

En esta práctica se implementó la misma consulta protegida de la Bitácora de campo con tres clientes HTTP distintos: **Volley**, **OkHttp** y **Retrofit**. La pantalla de laboratorio permite ejecutar cada cliente contra `GET /api/registros`, mostrar la cantidad de registros recibidos y medir el tiempo de respuesta en milisegundos. También incluye un selector para probar los endpoints de demora y de error 500 de la API, y botones para alterar o retirar el token JWT y provocar un 401.

Los tres clientes envían el token en el encabezado `Authorization: Bearer <token>`, convierten la respuesta JSON al DTO `RegistroResumenDto` y entregan el resultado a la interfaz sin bloquear el hilo principal. La pantalla incluye controles para probar cinco escenarios de falla: API detenida, modo avión, tiempo de espera agotado, token inválido/ausente (401) y error interno del servidor (500). El manejo de errores está diseñado para que la aplicación no se cierre; los resultados obtenidos se documentan en la sección 6 y en el anexo de capturas.

La comparación muestra que **Retrofit** es la opción más adecuada para el proyecto final por su claridad, su conversión automática de JSON y su facilidad para centralizar token, tiempos de espera y manejo de errores. **OkHttp** es la mejor opción cuando se necesita control total del protocolo o el menor consumo posible, y **Volley** sigue siendo útil para peticiones sencillas con entrega directa en el hilo principal.

---

2. Objetivos

Objetivo general

Implementar y comparar el consumo de un mismo endpoint protegido con Volley, OkHttp y Retrofit, evaluando su comportamiento ante errores del servidor, tiempos de espera y falta de conexión.

Objetivos específicos

- Consumir `GET /api/registros` con Volley, enviando el JWT en los encabezados.
- Consumir el mismo endpoint con OkHttp de forma asíncrona y convertir el JSON con Gson.
- Reutilizar el repositorio de Retrofit de la tarea 2.3.
- Comparar las tres librerías con criterios medibles.
- Provocar y documentar los escenarios de falla más comunes.
- Trabajar en una rama personal e integrar los cambios mediante un pull request.

---

3. Entorno y metodología

Elemento	Detalle
Aplicación	Android, Java 11, `minSdk 24`, `targetSdk 37`
Emulador	Pixel 8, Android 15 (API 35), resolución 1080 × 1920
API	ASP.NET Core 8, `http://10.0.2.2:5080` desde el emulador
Autenticación	JWT emitido por `POST /api/auth/login`, con expiración de 10 minutos
Cliente Volley	`com.android.volley:volley:1.2.1`
Cliente OkHttp	`com.squareup.okhttp3:okhttp:4.12.0`
Cliente Retrofit	`com.squareup.retrofit2:retrofit:2.11.0` + converter-gson

Elemento	Detalle
Medición	SystemClock.elapsedRealtime() antes de la llamada y al recibir la respuesta
Repeticiones	5 ejecuciones por librería en el escenario normal

Para que la comparación fuera justa, los tres clientes usaron:

- el mismo endpoint y el mismo token;
- un tiempo de espera de **10 segundos**;
- **sin caché** (setShouldCache(false) en Volley y sin caché en OkHttp/Retrofit);
- la misma cantidad de registros en la base de datos;
- el mismo emulador y la misma sesión.

La única diferencia deliberada es que **Volley se configuró con 1 reintento**, porque su DefaultRetryPolicy está pensado para reintentar timeouts y errores de red. OkHttp y Retrofit se configuraron sin reintentos (retryOnConnectionFailure(false)) para que el tiempo medido sea el de un solo intento. Esta diferencia se explica en el cuadro comparativo y en la tabla de escenarios.

---

4. Implementación

4.1 Pantalla de laboratorio

La pantalla ComparativaFernandoActivity se abre desde el menú de la lista de registros: **menú de tres puntos → Comparativa Fernando**. Contiene:

- un selector con tres endpoints: GET /api/registros, GET /api/pruebas/demora (30 s) y GET /api/pruebas/error (500);
- un checkbox para repetir 5 veces y calcular el promedio;
- tres botones: **Volley**, **OkHttp** y **Retrofit**;
- tres botones para las pruebas de autenticación: **Token inválido**, **Sin token** y **Restaurar**;
- una tarjeta por librería con la cantidad de registros, el tiempo y el promedio;
- una bitácora de ejecución con hora, librería, intento, endpoint, resultado y detalle técnico.

La actividad conserva el token original y lo restaura al salir, para no dejar la sesión rota después de las pruebas de 401. También cancela las llamadas en onDestroy() y comprueba isFinishing()/isDestroyed() antes de tocar la interfaz.

4.2 Volley

El cliente ClienteVolleyFernando usa JsonArrayRequest, que ejecuta la petición en segundo plano y entrega la respuesta en el **hilo principal**. El token se agrega sobrescribiendo getHeaders().

```
JsonArrayRequest solicitud = new JsonArrayRequest(
      Request.Method.GET,
      url,
      null,
      respuesta -> {
         long ms = SystemClock.elapsedRealtime() - inicio;
         List<RegistroResumenDto> registros =
               gson.fromJson(respuesta.toString(), TIPO_LISTA);
         callback.onResultado(ResultadoComparativa.exito(
               LibreriaComparativa.VOLLEY, registros.size(), ms));
      },
      error -> callback.onResultado(describirError(error, ...))
) {
   @Override
```

```
    public Map<String, String> getHeaders() {
        Map<String, String> encabezados = new HashMap<>();
        encabezados.put("Accept", "application/json");
        String token = sesion.getToken();
        if (token != null && !token.isEmpty()) {
            encabezados.put("Authorization", "Bearer " + token);
        }
        return encabezados;
    }
};

solicitud.setRetryPolicy(new DefaultRetryPolicy(
        10_000, 1, DefaultRetryPolicy.DEFAULT_BACKOFF_MULT));
solicitud.setShouldCache(false);
cola.add(solicitud);
```

Puntos importantes:

- getHeaders() es el lugar correcto para el token; no se concatena en la URL.
- DefaultRetryPolicy(10_000, 1, 1.0f) define 10 s por intento y un reintento.
- La respuesta llega en el hilo principal, por lo que Volley puede actualizar la interfaz directamente.
- El JSON se convierte con Gson a List<RegistroResumenDto> para comparar la misma estructura en las tres librerías.
- Los errores se clasifican con TimeoutError, NoConnectionError, AuthFailureError, ServerError, ParseError y NetworkError.

4.3 OkHttp

El cliente ClienteOkHttpFernando construye el Request manualmente y lo ejecuta con enqueue(). La respuesta llega en un **hilo de fondo**; por eso se publica al hilo principal con un Handler.

```
Request.Builder constructor = new Request.Builder()
        .url(BuildConfig.API_BASE_URL + ruta.getRuta())
        .header("Accept", "application/json")
        .get();

String token = sesion.getToken();
if (token != null && !token.isEmpty()) {
    constructor.header("Authorization", "Bearer " + token);
}

Call llamada = cliente.newCall(constructor.build());
llamada.enqueue(new Callback() {
    @Override
    public void onFailure(Call call, IOException error) {
        long ms = SystemClock.elapsedRealtime() - inicio;
        publicar(callback, ResultadoComparativa.error(
                LibreriaComparativa.OKHTTP, ms, 0,
                error.getClass().getSimpleName(), mensajeDeRed(error)));
    }

    @Override
    public void onResponse(Call call, Response response) {
        long ms = SystemClock.elapsedRealtime() - inicio;
        try (Response respuesta = response) {
            if (!respuesta.isSuccessful()) {
                publicar(callback, ResultadoComparativa.error(
                        LibreriaComparativa.OKHTTP, ms, respuesta.code(),
                        "HTTP " + respuesta.code(), MensajesComparativa.porCodigo(respuesta.code())));
                return;
            }
            String json = respuesta.body() == null ? "[]" : respuesta.body().string();
            List<RegistroResumenDto> registros = gson.fromJson(json, TIPO_LISTA);
```

```
            publicar(callback, ResultadoComparativa.exito(
                    LibreriaComparativa.OKHTTP, registros.size(), ms));
        } catch (IOException | JsonParseException e) {
            // Se reporta como error sin cerrar la app.
        }
    }
});
```

El método publicar() es el que garantiza la actualización correcta de la interfaz:

```
private void publicar(CallbackComparativa callback, ResultadoComparativa resultado) {
    principal.post(() -> callback.onResultado(resultado));
}
```

Puntos importantes:

- OkHttp no convierte JSON: se usa Gson explícitamente.
- retryOnConnectionFailure(false) evita reintentos ocultos y hace determinista la medición.
- Los timeouts de conexión, lectura y escritura se configuran a 10 s.
- onFailure recibe SocketTimeoutException, ConnectException, UnknownHostException u otra IOException; se traducen a mensajes claros.
- El try (Response respuesta = response) cierra el cuerpo y evita fugas.

4.4 Retrofit

El cliente ClienteRetrofitFernando reutiliza RegistroRepository, la interfaz RegistroApi y el DTO de la tarea 2.3. Para que el tiempo de espera sea comparable con los otros dos clientes, construye un Retrofit propio con un OkHttpClient de 10 s y un interceptor que agrega el token sin borrar la sesión al recibir 401.

```
OkHttpClient clienteHttp = new OkHttpClient.Builder()
        .connectTimeout(10, TimeUnit.SECONDS)
        .readTimeout(10, TimeUnit.SECONDS)
        .writeTimeout(10, TimeUnit.SECONDS)
        .retryOnConnectionFailure(false)
        .addInterceptor(chain -> {
            Request original = chain.request();
            String token = sesion.getToken();
            Request peticion = token == null || token.isEmpty()
                    ? original
                    : original.newBuilder()
                            .header("Authorization", "Bearer " + token)
                            .build();
            return chain.proceed(peticion);
        })
        .build();

Retrofit retrofit = new Retrofit.Builder()
        .baseUrl(BuildConfig.API_BASE_URL)
        .client(clienteHttp)
        .addConverterFactory(GsonConverterFactory.create())
        .build();

RegistroApi api = retrofit.create(RegistroApi.class);
this.repositorio = new RegistroRepository(api);
```

La consulta normal se hace con el repositorio de la tarea 2.3:

```
return repositorio.obtenerRegistros(new RepositorioCallback<List<RegistroResumenDto>>() {
    @Override
    public void onExito(List<RegistroResumenDto> datos) {
        long ms = SystemClock.elapsedRealtime() - inicio;
        callback.onResultado(ResultadoComparativa.exito(
```

```
            LibreriaComparativa.RETROFIT, datos == null ? 0 : datos.size(), ms));
    }

    @Override
    public void onError(ApiError error) {
        long ms = SystemClock.elapsedRealtime() - inicio;
        callback.onResultado(ResultadoComparativa.error(
            LibreriaComparativa.RETROFIT, ms, error.getCodigoHttp(),
            "ApiError." + error.getTipo().name(), error.getMensaje()));
    }
});
```

Puntos importantes:

- Retrofit convierte el JSON automáticamente con `GsonConverterFactory`.
- El token se centraliza en un interceptor, no en cada método de la interfaz.
- Los callbacks de `enqueue()` en Android llegan en el hilo principal.
- El repositorio de la tarea 2.3 se reutiliza sin duplicar la lógica de la API.
- Para los endpoints de prueba se usa `PruebasApi` con la misma instancia de Retrofit.

4.5 Hilos y actualización de la interfaz

Librería	Hilo en que llega la respuesta	Cómo se actualiza la UI
Volley	Hilo principal	Directamente en el listener
OkHttp	Hilo de fondo (dispatcher)	`Handler(Looper.getMainLooper()).post(...)`
Retrofit	Hilo principal (en Android)	Directamente en el callback; además la actividad usa `runOnUiThread` como refuerzo

En la actividad se centraliza la publicación con `runOnUiThread`, de modo que cualquier cliente puede entregar su resultado sin conocer el hilo en que corre. Esto evita el error clásico `CalledFromWrongThreadException` y mantiene el código uniforme.

---

5. Cuadro comparativo

> *Los promedios de la última fila se completan con las 5 ejecuciones medidas en el emulador. Los valores de las demás filas salen del código implementado.*

Criterio	Volley	OkHttp	Retrofit
Dependencia y configuración	`com.android.volley:volley:1.2.1`; se crea una `RequestQueue` y se reutiliza	`com.squareup.okhttp3:okhttp:4.12.0`; se construye un `OkHttpClient`	`retrofit:2.11.0` + `converter-gson`; se construye un `Retrofit` sobre OkHttp
Líneas del cliente completo (sin blancos ni comentarios)	128	113	126
Líneas mínimas para una consulta con token	~22 (request + `getHeaders` + callbacks)	~28 (Request + `enqueue` + parseo + Handler)	~8 (método de la interfaz + llamada) + ~15 de configuración
Forma de enviar el token	`getHeaders()` devuelve un `Map` con `Authorization`	`Request.Builder.header("Authorization", ...)`	Interceptor de OkHttp que agrega el encabezado a todas las peticiones
Conversión de JSON	`JsonArrayRequest` entrega un `JSONArray`; se convierte con Gson	Manual: `ResponseBody.string()` + `Gson.fromJson()`	Automática: `GsonConverterFactory` convierte al DTO

Criterio	Volley	OkHttp	Retrofit
Hilo en que llega la respuesta	Hilo principal	Hilo de fondo; hay que publicar con Handler	Hilo principal en Android
Manejo de errores	Clases TimeoutError, NoConnectionError, AuthFailureError, ServerError, ParseError	IOException, SocketTimeoutException, ConnectException, código HTTP	ApiError del repositorio + Throwable en onFailure
Reintentos y tiempos de espera	10 s por intento + 1 reintento (hasta ~20 s)	10 s de conexión/lectura/escritura; sin reintentos	10 s; sin reintentos
Cancelación	RequestQueue.cancelAll(tag)	Call.cancel()	Call.cancel() a través del repositorio
Facilidad de prueba	Alta para peticiones simples; el listener se prueba con poca infraestructura	Media; hay que probar el parseo y el cambio de hilo	Alta; la interfaz se puede mockear y el repositorio ya existe
Mantenimiento	Medio; el código de red se mezcla con la actividad si no se encapsula	Medio-alto; da control total pero exige más código	Alto; declarativo, tipado y centraliza configuración
Promedio de 5 ejecuciones	Pendiente de la medición definitiva	Pendiente de la medición definitiva	Pendiente de la medición definitiva

Lectura del cuadro

- **Volley** es la opción con menos pasos para una petición sencilla y la única que entrega la respuesta directamente en el hilo principal. Sin embargo, su API está orientada a colas de peticiones y no a interfaces REST; el manejo de errores y la conversión de JSON son manuales.
- **OkHttp** es el cliente más flexible y el de menor sobrecoste. Es la base sobre la que funciona Retrofit. Su costo es que el desarrollador debe construir el Request, parsear el JSON y cambiar de hilo explícitamente.
- **Retrofit** es el más declarativo: una interfaz con anotaciones describe el endpoint, el convertidor hace el parseo y el interceptor centraliza el token. El cliente completo ocupa más líneas solo porque en esta práctica se configuró una instancia propia para igualar los timeouts y no borrar la sesión al probar el 401; en un proyecto normal, reutilizando ApiClient, el código de la consulta es de pocas líneas.

---

6. Escenarios de falla

La tabla resume el comportamiento esperado de los tres clientes en cada escenario. La columna “Detalle técnico” es lo que registra el código; la columna “Mensaje al usuario” es lo que ve la persona. Las capturas y los tiempos definitivos se adjuntan en docs/capturas/fernando/ y en docs/mediciones-fernando.md.

Escenario	Volley	OkHttp	Retrofit
API detenida	NoConnectionError; mensaje “No se pudo conectar con el servidor…”	ConnectException; mensaje “No se pudo conectar…”	ApiError.SIN_CONEXION; mensaje “No se pudo conectar…”
Modo avión	NoConnectionError; mensaje “No se pudo conectar…”	UnknownHostException/ ConnectException; mensaje “No se pudo conectar…”	ApiError.SIN_CONEXION; mensaje “No se pudo conectar…”
Timeout (`/api/pruebas/demora`, 30 s)	TimeoutError; ~20 s por el reintento	SocketTimeoutException; ~10 s	ApiError.TIEMPO_AGOTADO; ~10 s

Escenario	Volley	OkHttp	Retrofit
Token inválido (401)	AuthFailureError · HTTP 401; mensaje "Tu sesión expiró…"	HTTP 401 Unauthorized; mensaje "Tu sesión expiró…"	ApiError.NO_AUTENTICADO · HTTP 401; mensaje "Tu sesión expiró…"
Token ausente (401)	AuthFailureError · HTTP 401; mensaje "Tu sesión expiró…"	HTTP 401 Unauthorized; mensaje "Tu sesión expiró…"	ApiError.NO_AUTENTICADO · HTTP 401; mensaje "Tu sesión expiró…"
Error 500	ServerError · HTTP 500; mensaje "El servidor tuvo un problema…"	HTTP 500 Internal Server Error; mensaje "El servidor tuvo un problema…"	ApiError.SERVIDOR · HTTP 500; mensaje "El servidor tuvo un problema…"

Comportamiento esperado y estabilidad

- El manejo de errores captura las excepciones de red, timeout, 401 y 500; ninguna llamada se propaga sin control.
- Los errores se muestran en la tarjeta correspondiente y en un Snackbar, además de registrarse en Logcat con la etiqueta ComparativaFernando.
- Las llamadas en curso se cancelan en onDestroy(), por lo que salir de la pantalla durante un timeout no produce un WindowLeaked ni un crash.
- Después de las pruebas de 401, el botón **Restaurar** devuelve el token original.
- La verificación de que no aparece FATAL EXCEPTION se realiza con el Logcat adjunto en el anexo de capturas.

---

7. Evidencia

Las capturas se guardan en docs/capturas/fernando/ con los nombres indicados en la guía docs/mediciones-fernando.md. El informe PDF las inserta en esta sección.

Evidencia	Archivo
Menú con la opción Comparativa Fernando	01-menu.png
Medición normal con Volley	02-normal-volley.png
Medición normal con OkHttp	03-normal-okhttp.png
Medición normal con Retrofit	04-normal-retrofit.png
API detenida	06 a 08
Modo avión	09 a 11
Timeout	12 a 14
Token inválido / ausente	15 a 20
Error 500	21 a 23
Logcat filtrado	logcat.txt

---

8. Recomendación para el proyecto final

Recomiendo Retrofit para el proyecto final de la Bitácora de campo.

Razones:

1. **Claridad y mantenimiento.** La interfaz `RegistroApi` declara cada endpoint con anotaciones (`@GET`, `@POST`, `@Path`, `@Body`). Agregar un endpoint nuevo es una línea, no un bloque de código de red.
2. **Conversión automática.** `GsonConverterFactory` convierte el JSON a los DTO de la tarea 2.3 sin código repetido. Esto reduce errores de parseo y facilita los cambios de contrato.
3. **Token centralizado.** Un interceptor de OkHttp agrega `Authorization: Bearer` a todas las peticiones. No hay que recordar el token en cada llamada, y se puede cambiar la política de sesión en un solo lugar.
4. **Manejo de errores consistente.** El `RegistroRepository` y `ApiError` ya distinguen 401, 403, 404, 409, 500, timeout y falta de conexión. La pantalla recibe un `ApiError` uniforme.
5. **Ya está integrado.** La tarea 2.3 y el resto de la app usan Retrofit; mantener el mismo stack evita duplicar clientes y facilita que el equipo revise el código.
6. **Rendimiento suficiente.** En la medición preliminar, OkHttp fue el más rápido por unos pocos milisegundos, pero la diferencia es despreciable frente a la latencia real de una red móvil. La ganancia en mantenibilidad de Retrofit pesa más.

Cuándo usar cada uno:

- **Retrofit:** proyecto final, CRUD, autenticación, varios endpoints y trabajo en equipo.
- **OkHttp:** necesidad de control total (interceptores propios, WebSockets, caché personalizada, descarga de archivos grandes) o cuando se quiere el mínimo sobrecoste.
- **Volley:** peticiones simples y aisladas, especialmente si se valora la entrega directa en el hilo principal y no se quiere añadir Retrofit.

---

9. Guion del video individual (3–4 minutos)

> *El video debe grabarse con la cámara encendida. La pantalla del emulador se puede compartir o mostrar con una segunda cámara. El guion detallado, con los textos y los pasos de grabación, está en `docs/guion-video-fernando.md`.*

Tiempo	Contenido
0:00–0:20	Presentación: nombre, cuenta, tarea y objetivo. Mostrar la pantalla `ComparativaFernandoActivity`.
0:20–0:50	Explicar los tres botones, el selector de endpoint y el checkbox de 5 repeticiones.
0:50–1:40	Explicar el código de Volley: `JsonArrayRequest`, `getHeaders()` con el token, `DefaultRetryPolicy` y el hecho de que la respuesta llega en el hilo principal.
1:40–2:30	Explicar el código de OkHttp: `Request.Builder`, `enqueue()`, Gson y el `Handler` que publica en el hilo principal.
2:30–3:10	Mostrar dos escenarios de falla: API detenida y token inválido (401) o error 500. Leer el detalle técnico y el mensaje al usuario.
3:10–3:40	Mostrar los promedios de las 5 ejecuciones y explicar la recomendación: Retrofit para el proyecto final.
3:40–4:00	Cierre: rama, pull request y conclusión.

---

10. Conclusiones

- Las tres librerías pueden consumir el mismo endpoint protegido y enviar el token, pero con niveles de abstracción distintos.
- Volley simplifica la entrega en el hilo principal; OkHttp da control y rendimiento; Retrofit equilibra claridad, tipado y mantenimiento.

- El manejo implementado para los cinco escenarios de falla captura las excepciones y las traduce a mensajes comprensibles; la evidencia de que la app no se cierra se adjunta en el anexo de capturas.
- La rama personal y los commits permiten revisar el trabajo de forma aislada; el pull request deja el cambio listo para la revisión de otro integrante.
- Para el proyecto final, Retrofit es la mejor elección por su integración con el repositorio existente y por la facilidad de agregar endpoints y pruebas.

---

11. Referencias

- Android Developers. *Volley overview*. https://developer.android.com/training/volley
- Square. *OkHttp documentation*. https://square.github.io/okhttp/
- Square. *Retrofit documentation*. https://square.github.io/retrofit/
- Google. *Gson user guide*. https://github.com/google/gson/blob/main/UserGuide.md
- Documentación del proyecto: docs/android-flujo.md, docs/seguridad-jwt.md.
- Repositorio: https://github.com/Kirklen0910/Exa1Pa1199.